

Topic 3 降维技术
Part 3：局部保持映射（LPP）

魏准
浙江大学，信息与电子工程学院
2026

总体内容

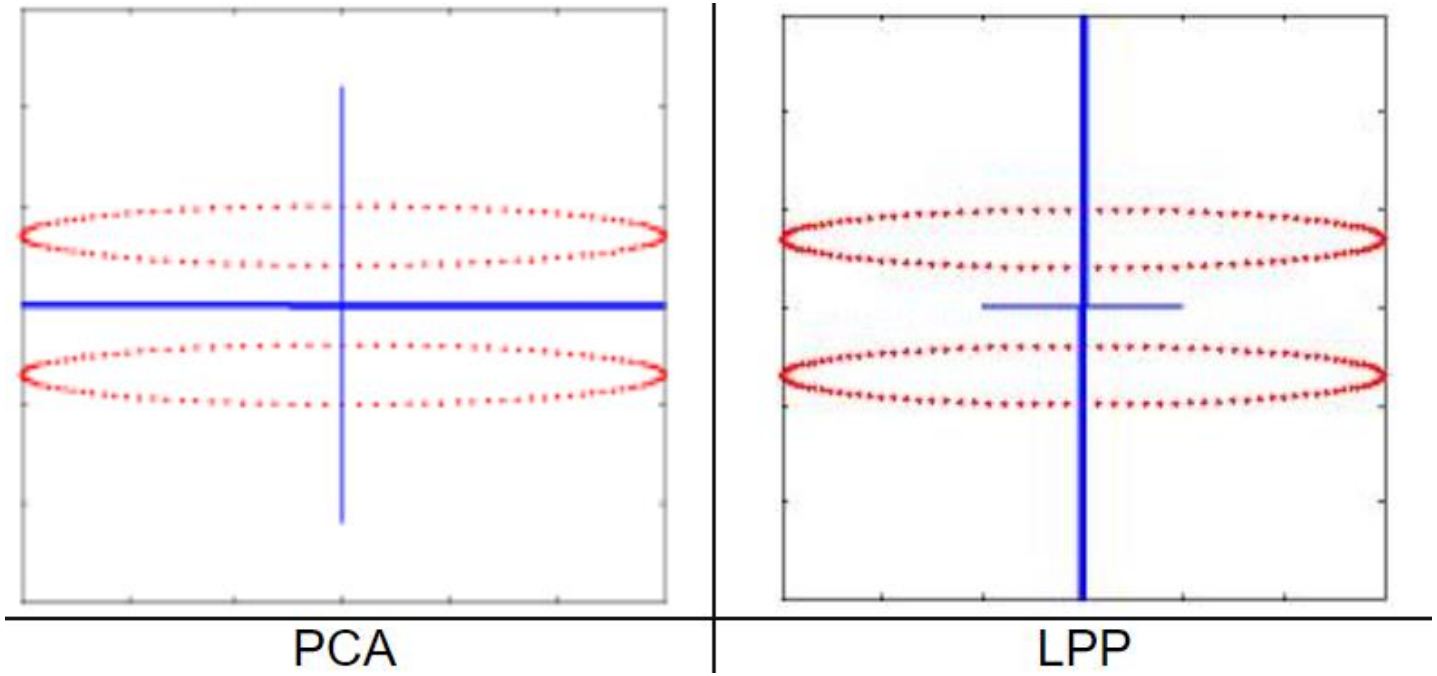
主要知识点

- 局部保持投影(Locality Preserving Projections，LPP)
- 实验3-4；
- LPP代码实现解析；

1.1：局部保持投影(Locality Preserving Projections，LPP)

LPP:

- 降维的同时保留空间中样本的**局部邻域结构**；避免样本集的发散，保持原来的**近邻结构**;



第一个投影方向为长蓝线，第二个投影方向为短蓝线；若降到一维则PCA将导致两个圆的信息完全重合，丢失了大部分信息。

He, X. and Niyogi, P., 2003. Locality preserving projections. Advances in neural information processing systems, 16(16), pp.153-160.

1.1 LPP解析

目标：给定在R^n空间的$x_1, x_2, x_3, \ldots, x_m$，通过映射矩阵$A$，将m个点映射到$R^{n'}$空间的$y_1, y_2, y_3, \ldots, y_m$，即$y_i = Ax_i$。满足$n' < n$且降维的同时保留空间中样本的**局部邻域结构**；

目标函数

$$\sum_{ij}(y_i - y_j)^2 W_{ij} \qquad W_{ij} = e^{-\frac{\|\mathbf{x}_i - \mathbf{x}_j\|^2}{t}}$$

- 设定如果x_i, x_j离的比较远（不是近邻关系），则$W_{i,j}$直接设为0，目标函数为0；（注意设置$W_{i,i} = 0$）；
- 设定如果x_i, x_j离的比较近（是近邻关系），则$W_{i,j}$比较大，故需要保证y_i, y_j也要近，才能使目标函数小；

假设a是变换矩阵，即$y = a^T X$。

$$\frac{1}{2}\sum_{ij}(y_i - y_j)^2 W_{ij} = \frac{1}{2}\sum_{ij}(\mathbf{a}^T\mathbf{x}_i - \mathbf{a}^T\mathbf{x}_j)^2 W_{ij}$$

$$= \sum_i \mathbf{a}^T\mathbf{x}_i D_{ii}\mathbf{x}_i^T\mathbf{a} - \sum_{ij}\mathbf{a}^T\mathbf{x}_i W_{ij}\mathbf{x}_j^T\mathbf{a} = \mathbf{a}^T X(D-W)X^T\mathbf{a} = \mathbf{a}^T XLX^T\mathbf{a}$$

- 满足X=$[x_1, x_2, x_3, \ldots, x_m]$且D是对角矩阵，且满足$D_{ii} = \Sigma_j \mathrm{W}_{\mathrm{i},j}$；
- L= D − W为拉普拉斯矩阵

由于D_{ii}矩阵越大，其对应的y_i所占比重（重要性）越高，因此我们设定一个约束条件：

$$\mathbf{y}^T D\mathbf{y} = 1 \Rightarrow \mathbf{a}^T XDX^T\mathbf{a} = 1$$

$$y_i = a^T x_i$$

$$y_1 = \begin{bmatrix} y^1 \\ y^2 \\ \vdots \\ y^{n'} \end{bmatrix}_{n' \times 1} = \begin{bmatrix} \quad \end{bmatrix}_{n' \times n} \begin{bmatrix} x^1 \\ x^2 \\ \vdots \\ x^n \end{bmatrix}_{n \times 1}$$

$$[y_1, y_2, \cdots y_m]_{n' \times m} = a^T [x_1, x_2, \cdots x_m]$$

$$= [a^T x_1, a^T x_2, \cdots a^T x_m]$$

$$Loss = \min \frac{1}{2} \sum_{i,j} (y_i - y_j)^2 W_{ij} = \min \frac{1}{2} \sum_{i,j} (y_i - y_j)^T (y_i - y_j) W_{ij}$$

$$= \min \frac{1}{2} \left(\sum_{i,j} y_i^T y_i W_{ij} + \sum_{i,j} y_j^T y_j W_{ij} - \sum_{i,j} y_i^T y_j W_{ij} - \sum_{i,j} y_j^T y_i W_{ij} \right)$$

$$= \sum_i \left(\sum_j W_{ij} \right) y_i^T y_i - \sum_{i,j} y_i^T y_j W_{ij} = \sum_i D_{ii} y_i^T y_i - \sum_{i,j} (y_i^T y_j W_{ij})$$

for $D_{ii} = \sum_j W_{ij}$, 令 $X = [x_1, x_2, \cdots x_m]$, $D = \begin{bmatrix} D_{11} & & \\ & D_{22} & \\ & & \ddots \\ & & & D_{mm} \end{bmatrix}$

$$L = D - W \Rightarrow Loss = tr(a^T X L X^T a)$$

$$s.t. \quad tr(a^T X D X^T a) = 1$$

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

最后目标函数变为

$$Loss = tr(a^T X L X^T a)$$

$$s.t. \quad tr(a^T X D X^T a) = 1$$

利用拉格朗日乘子法：

$$XLX^T\mathbf{a} = \lambda XDX^T\mathbf{a}$$

- a的解为$(XDX^T)^{-1}XLX^T$的n'个最小非零特征值所对应的特征向量组成的矩阵；
- X^T a即为降维后的数据

实验3-4：

利用sklearn.datasets.load_digits函数，导入手写数字数据集作为X^T，通过LPP对生成的随机数据进行降维（n_dim=2），并可视化降维后的数据。

$$W_{ij} = e^{-\frac{\|\mathbf{X}_i - \mathbf{X}_j\|^2}{t}}$$

具体步骤：

- **Step 1**:定义函数-计算W（输入变量data, n_neighbour, t；输出变量W）；
- **Step 2**:定义函数- LPP （输入变量data，n’，n_neighbour，t；输出降维后的数据）
  - ✓ 计算D矩阵;
  - ✓ 计算L矩阵;
  - ✓ 计算XDXT;
  - ✓ 计算XLXT;
  - ✓ $(XDX^T)^{-1}XLX^T$特征值分解;
  - ✓ 找出$(XDX^T)^{-1}XLX^T$的n’个最小非零特征值所对应的特征向量
  - ✓ 计算降维后的数据
- **Step 3**:编写测试函数- 通过sklearn.datasets.load_digits().data导入数据X、利用LPP函数降维;

1.2 LPP代码实现解析；

Step1定义函数-计算W
Step1.1:计算高维样本每一个元素之间的距离

```
def cal_pairwise_dist(x):  #cal_pairwise_dist: 计算高维样本之间的距离

    sum_x = np.sum(np.square(x), 1)  #np.square 返回一个新数组，其元素值为原数组元素平方;
    dist = np.add(np.add(-2 * np.dot(x, x.T), sum_x).T, sum_x)  #np.dot矩阵乘法。np.add 拓展为相同array后相加

    return dist #计算(a-b)^2 = a^2 + b^2 - 2*a*b
```

Step 1.2:计算W矩阵

```
def cal_W_ele(dist, t = 1.0):
    return np.exp(-(dist/t))  # calculate W elements

def cal_W(data, n_neigh = 10, t = 1):
    dist = cal_pairwise_dist(data)
    dist[dist < 0] = 0
    print('dist shape:', dist.shape)
    n = dist.shape[0]
    print('n:', n)
    W_dist = cal_W_ele(dist, t)  # 计算Welement

    W = np.zeros((n, n))
    for i in range(n):
        j_n = np.argsort(dist[i])[1:1 + n_neigh]  #元素从小到大排列
        W[i, j_n] = W_dist[i, j_n]   #除了前n_neigh个元素，W其他元素为0
    return W
```

Step 3:定义LPP函数

```
def LPP(data,
        n_dims = 2,  #target dimension
        n_neigh = 30, t = 1.0):
    W = XXX
    D = XXX
    L = XXX
    XDXT = XXX
    XLXT = XXX

    data_ndim = XXX  降维后的数据
    return data_ndim
```

Step 4:测试LPP函数

```
import matplotlib.pyplot as plt
from sklearn.datasets import load_digits
from LPP_Zhun import *
from numpy import *

X = load_digits().data  # 1797*64
y = load_digits().target  # Label 0, 1, ...

dist = cal_pairwise_dist(X) # 1797*1797
max_dist = np.max(dist)

print('max_dist:', max_dist)
data_LPP = LPP(X, n_dims = 2, n_neigh = 5, t = 0.01*max_dist)
```